czii-cryo-et-object-identification

7位解法 [ガウスヒートマップを使った3D-UNet]

Rank	7
Team	kobakos
Author	kk
Topic ID	561447
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561447
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

7位解法

まず、この興味深いコンペを開催してくださったコンペ主催者、データセット提供者、そして参加者の皆さんに心から感謝します。さまざまなモデルで実験するのはとても楽しく、このコンペで多くを学びました。これは私の7位の提出の解法です。

概要

私のアプローチでは、クラスごとのガウスヒートマップを予測する 3D セグメンテーションモデルを使いました。モデルはまずシミュレーションデータセットで事前学習し（1fold、wbp を検証に使用）、その後、実験データセットでファインチューニングしました（4 fold）。すべてのモデルは、モデルがより多くの予測を出すように、非常に高い pos_weight の重み付き BCE で学習しました。学習中は、Mixup、Cutmix、RandomFlip、Affine（xy 平面内の回転のみ、事前学習でのみ使用）、rot90（xy 平面内のみ）、その他の画素値のデータ拡張など、強いデータ拡張を行いました。最終提出は、3つのモデルスープ（それぞれ4つの fold を組み合わせたもの）のアンサンブルで、4x の test-time augmentation (TTA) と 4x の sliding window を適用しています。

前処理とデータ拡張

事前学習の時間を減らすため、シミュレーションデータセット全体を学習に使い、実験データセットの wbp 版を検証に使いました。

前処理は最小限で、パーセンタイルでのクリッピング（0.1–99.9%）と、データセットごとのスケーリング係数だけです。

- シミュレーション: ×1.0
- WBP: ×1e4
- Denoised: ×1e5

stride 64 の sliding window を使い、最後のウィンドウを除いて、experiment ごとに 1x8x8=64 個のウィンドウ（x と y は 0, 64, 128, …, 384, 448、z は 0）を作りました。そして学習データを生成するときに、クロップの座標をランダムにずらしました。

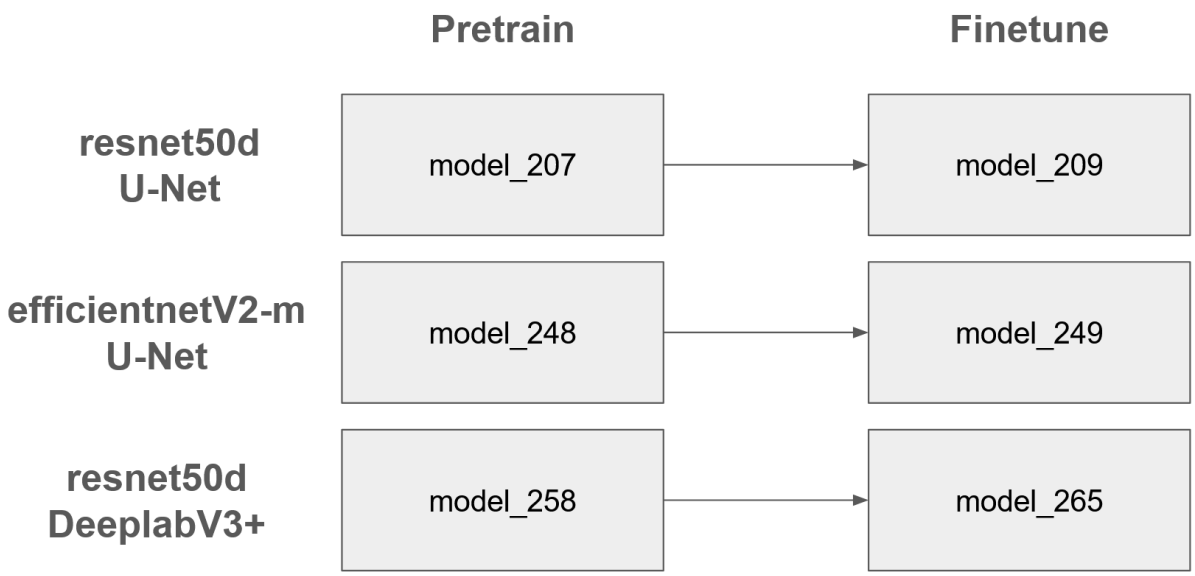
使ったデータ拡張は次のとおりです。

- shift（あらかじめ決めたクロップウィンドウから最大 64）
- cutmix
- mixup
- randomflip（全軸）
- rot90（xy 平面のみ）
- affine（各軸のスケール、回転は xy 平面のみ）※事前学習時のみ
- contrast（0.7 ~ 1.5）
- gamma（0.8 ~ 1.2）

- gaussian noise（0 ~ 0.05）※ファインチューニング時のみ

モデルのアーキテクチャ

最終提出では、U-Net ベースのモデル2つと DeepLab ベースのモデル1つ、合計3つのモデルを使いました。2つの U-net ベースのモデルのうち、1つは ResNet50d、もう1つは EfficientNetV2-M の backbone を持ちます。DeepLab ベースのモデルの backbone は ResNet50d です。位置推定の性能を上げるためにオフセット予測の head を持つモデルの学習も試しましたが、私の場合はうまくいかなかったので、最終提出では単純なセグメンテーションの head だけを使っています。モデル 207 以外のすべてのモデルで、decay 0.995 の ModelEMA を使いました。



5つの特徴ステージのうち最初の4つだけを使うと、同等かそれ以上の性能になったので、アンサンブル内の一部のモデル（resnet50d）ではそうしています。粒子の位置を特定するのに必要な情報が、高解像度の特徴マップに多く含まれていたからだと推測しています。

最終アンサンブルでは、[resnet50d, resnet50d, efficientnetv2-m] の backbone を使いました。

最終提出では使いませんでしたが、DeepLabV3+ も比較的高いスコアを出すことができ、推論時間は U-Net モデルの半分しかかかりませんでした。DeepLabV3+ を使ってより多くのアンサンブルを行えば、より良い提出を作れたかもしれません。

モデル	Backbone	skip connection の数	デコーダのチャネル	初期重み
Model 207 & 209	ResNet50d	4	[128, 64, 32, 16]	ra2_in1k
Model 248 & 249	EfficientNetV2-M	5	[512, 176, 80, 48, 24]	in21k_ft_in1k

モデル	Backbone	高解像度特徴の数	セグメンテーション特徴の数	初期重み
Model 258 & 265	ResNet50d	48	256	ra2_in1k

Loss 関数

loss 関数は重み付き BCE loss で、「難しい」粒子と、ターゲットが陽性である領域に、より大きな重みを与えています。loss 関数の式は次のとおりです。

$$

L_{c, i} = -w_c \left(p_c \cdot y_{c, i} \cdot \log(t_{c, i}) + (1 - y_{c, i}) \cdot \log(1 - t_{c, i}) \right)

$$

$$

Loss = \frac{1}{5N} \sum_{c=1}^{5} \sum_{i=1}^{N} L_{c, i}

$$

ここで (L_{c, i}) はクラス (c) とピクセル (i) の loss、(w_c) はクラス (c) の重み、(p_c) はクラス (c) の陽性ピクセルの重み、(y_{c, i}) はクラス (c) とピクセル (i) の予測、(t_{c, i}) はクラス (c) とピクセル (i) のターゲットです。(N) は画像のピクセル数で、和はすべてのピクセルとクラスにわたってとります。

クラスの重みと陽性の重みを下に示します。

クラスの順序は [apo-ferritin, beta-galactosidase, ribosome, thyroglobulin, virus-like-particle] です。

モデル	クラスの重み	陽性の重み
Model 207	[1, 2, 1, 2, 1]	[28, 54, 36, 48, 24]
Model 209	[1, 2, 1, 2, 1]	[54, 96, 24, 36, 36]
Model 248	[1, 2, 1, 2, 1] × 5/7	[24, 36, 36, 48, 20]
Model 249	[1, 2, 1, 2, 1] × 5/7	[54, 96, 36, 36, 36]
Model 258	[1, 3, 2, 3, 1]	[42, 48, 72, 85, 54]
Model 265	[1, 3, 2, 3, 1]	[54, 96, 24, 36, 36]

推論

推論は、サイズ (192, 128, 128) にパディングしたボリュームで行いました。[184, 630, 630] のトモグラムから、x 軸と y 軸で stride 64 の 81 個のウィンドウを作り、その結果 4x の TTA になります。結果を1つのヒートマップにまとめるときは、sigmoid をかけたヒートマップを平均するより、logit を平均する方が良い結果でした。

端のアーティファクトを減らすため、端に近づくほど下がる傾斜のついた重み関数も適用しました。

TTA として反転のデータ拡張を行いました。モデルの数に応じて、8x から 3x（normal、flipx、flipy）の TTA を適用しました。

後処理

最初は CCL + 重心を使って座標を決めていましたが、局所最大の検出に maxpool を使う方法に切り替えました。ノイズを減らすため、maxpool の前に kernel size 5、sigma 1.0 の gaussian blur をかけ、局所最大の検出の後に

radius=0.5*particle_radius の weighted box fusion を適用しました。maxpool の方法を使うと、性能の低下を最小限（約 0.002 の低下）に抑えつつ推論速度が大幅に改善し、より多くの TTA とアンサンブルが可能になりました。

pos_weight が非常に高いため、閾値を 0.5 前後にするだけで、モデルは多くの FP を予測するようになりました。最終提出では、logit 空間で [0.3, 0.2, -0.2, -0.2, -0.2] の閾値を使いました。

アンサンブル戦略

異なるモデルの出力をアンサンブルするときは、sigmoid をかけたヒートマップを平均したり、最終予測に WBF を適用したりするより、ヒートマップの logit を平均する方が良い結果でした。3つのモデルで 4x Flip TTA、4x Sliding Window にするのが、TTA とアンサンブルの最良のバランスのようでした。

LB の probing

対象のクラスだけで次の3つの提出をすると、

- 通常
- FP を追加（各予測に (0, 0, 0) * 20 を付け足すなど）
- 予測を2倍にする

連立方程式を解いて TP、FP、FN の数を計算できます。

```
mat = np.array()
TP, FP, FN = np.linalg.solve(mat, )
```

粒子の総数が分かれば、2つの提出から TP、FP、FN を計算できます。（係数行列の最下行を 1, 0, 1 に、ターゲットを 0, -added_fp*more_fp_score, num_particles に変更します。）

最初はこの情報を使ってモデルを調整するのが怖かったのですが、probing で得た粒子数（TP + FN）が論文に記載された平均の数と一致したので、private のテストセットも public のテストセットと同じ粒子の分布を持つと仮定しました。コンペの前半は LB に過学習しないように提出数を抑えようとしていましたが、この後は自分の CV より LB を信頼することにしました。

スコア

説明	Public Score	Private Score
Model 209	0.77320	0.76665
Model 249	0.77024	0.76478
Model 265	0.76824	0.76057
3つのアンサンブル	0.78351	0.77708